

DOCUMENTATION

ABSTRACT:

This project presents the design and implementation of a wireless interactive display system using an ESP32 microcontroller and multiple MAX7219-driven display modules. The system creates a digital drawing platform where users can sketch on a web-based canvas hosted by the ESP32 operating in Access Point mode. Drawing coordinates captured through the browser are processed in real time and translated into corresponding LED segment activations on the physical display array. The architecture combines embedded hardware, wireless communication, and web technologies to establish seamless interaction between a remote user device and the display hardware. By utilizing daisy-chained MAX7219 drivers, the system achieves efficient control of multiple display units while minimizing microcontroller pin usage. The project demonstrates practical applications of IoT-enabled visualization, real-time data transfer, and human-machine interaction through an innovative display interface.

List of figures:

S.No	Figure	Page.no
Fig.1	Pin Diagram of MAX7219	2
Fig.2	Circuit Diagram and arrangement	3
Fig.3	ESP32 pin diagram	4
Fig.4	4x8 integration	7
Fig.5	Final Output	11

List of abbreviations:

Abbreviation	Full form
IoT	Internet of things
LED	Light emitting diode
MCU	Micro-controller unit
AP mode	Access point mode
DC	Direct current
GPIO	General purpose input/output

Notation:

Notation	Representation
V	Volts
mV	milli Volts
A	Ampere
mA	milli Ampere
Ω	ohms
MHz	Mega Hertz

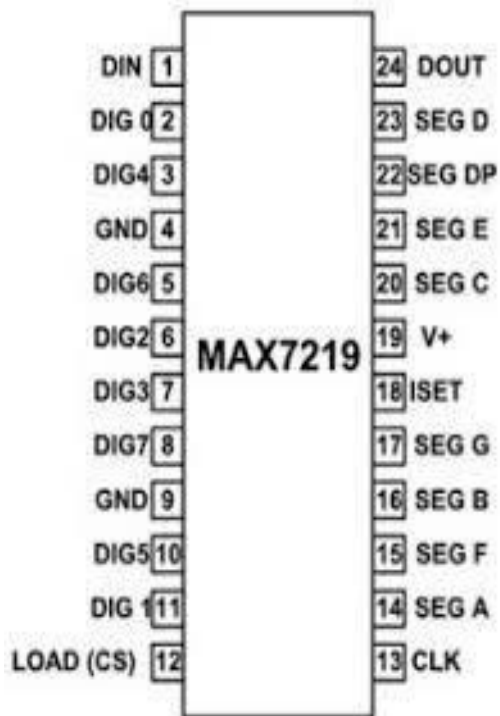


Fig.1. Pin Diagram of MAX7219

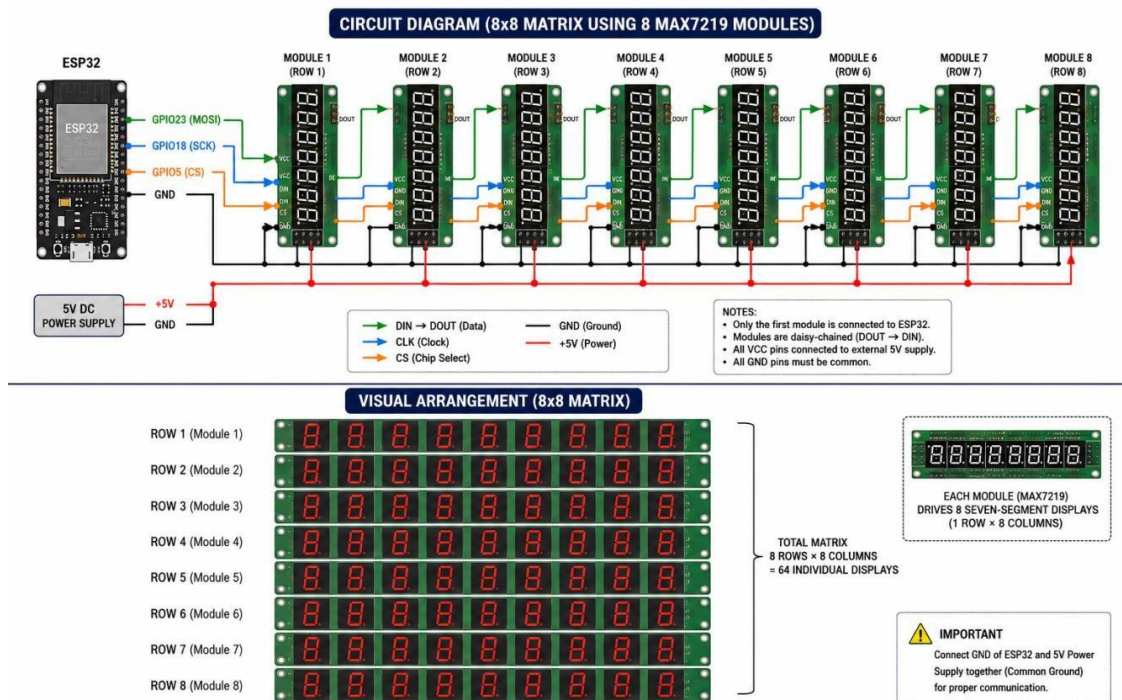


Fig.2. Circuit Diagram and arrangement

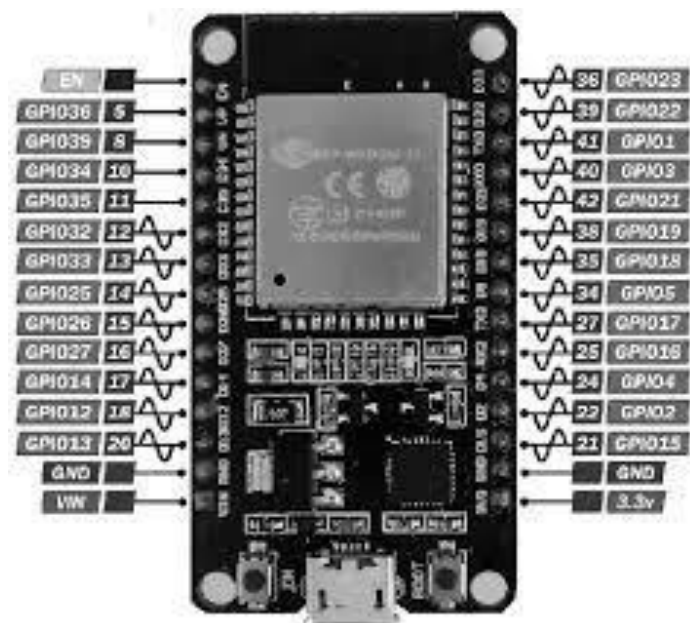


Fig.3.ESP32 pin diagram

Design Choice:

A 5V, 2A or 3A DC power supply is selected for safe and stable operation.

INTRODUCTION:

1.1. Background and Motivation Displays are crucial components in almost all electronic systems because they provide direct visual feedback to the user. While there are many types of high-resolution screens available today, we chose to use standard 7-segment displays for this project. Our motivation is to take a traditional, basic electronic component and use it in a creative way. By building a digital canvas out of 7-segment displays, we are creating a unique, retro-style interactive art board while gaining practical experience with hardware.

1.2. System Overview A single 7-segment display contains 8 individual LEDs (seven main segments plus one decimal point). For our project, we are using 64 of these displays arranged in an 8x8 matrix. To control this many LEDs without running out of microcontroller pins, we grouped them into modules. Each module consists of eight 7-segment displays driven by one MAX7219 IC. We daisy-chained 8 of these modules together to build the full 64-display matrix.

To act as the brain of the system, we chose the ESP32 microcontroller because of its fast 240 MHz clock speed and built-in Wi-Fi capabilities. Because driving 64 displays draws a significant amount of current, we cannot power the LEDs directly from the ESP32. To prevent damaging the board, we provided a separate, dedicated 5V power supply just for the display modules.

1.3. IoT and Wireless Connectivity The core idea of this project is to allow a user to control the physical display wirelessly using a remote device, such as a smartphone or laptop. To achieve this, the user interacts with a digital drawing canvas on their screen. The drawing data is transmitted over a local Wi-Fi connection to the ESP32. The ESP32 then processes the incoming data and sends the exact LED mapping instructions to the MAX7219 drivers, which lights up the physical displays to match the drawing.

1.4. Team Workflow and Project Goals Our team consists of three members, and we divided the project into three distinct tracks to manage the workload efficiently. The work was split into embedded hardware (wiring and assembling

the physical circuit), firmware (writing the C++ logic for the ESP32), and IoT (developing the web interface and wireless data link). Our ultimate goal is to successfully combine these three areas to build a fully functional, integrated device.

SYSTEM DESIGN AND IMPLEMENTATION:

Required Components

- ESP-32 Microcontroller
- 8 × MAX7219 LED Display Driver Modules
- 9V DC Power Supply
- 5V DC Buck Converter
- Jumper Wires
- Remote User Device (Smartphone/Laptop)

Hardware Architecture:

The ESP-32 serves as the central processing unit for the system. To minimize the required GPIO pins, the eight MAX7219 display modules are connected in a serial daisy-chain configuration using standard jumper wires.

Power distribution is a critical component of the architecture. The MAX7219 modules are powered via a dedicated 5V buck converter, which steps down the primary 9V power supply. The display array is intentionally powered externally rather than through the ESP-32's onboard voltage regulator. This design choice isolates the microcontroller from the high current demands of the LEDs, ensuring system stability and preventing voltage drops or hardware brown-outs.

Software Configuration:

The system firmware was developed and flashed using the Arduino IDE. The ESP-32 is programmed to operate in Access Point (AP) mode, allowing remote

devices to connect directly to the local network without requiring an external internet router. Once connected, the ESP-32 serves a custom-built webpage containing an interactive drawing canvas, which captures and transmits user input to the microcontroller.

Power Calculations:

A power analysis was conducted to justify the use of the external buck converter and to ensure safe operating limits for the display drivers:

- **Base LED Draw:** Each LED segment draws approximately **20 mA**.
- **Total LEDs:** Each 7-segment display contains 8 LEDs (7 segments + 1 decimal point). Across 64 displays, the system contains exactly 512 individual LEDs.
- **Estimated Dynamic Load:** During standard canvas interactions, not all segments are illuminated simultaneously. Assuming an average activation of 3 segments per display, the estimated dynamic current load is calculated as: $3 \text{ segments} \times 20 \text{ mA} \times 64 \text{ displays} = 3.84 \text{ A}$.
- **Driver Limits:** The maximum theoretical current per MAX7219 driver is **160 mA** ($8 \text{ segments} \times 20 \text{ mA}$). For the eight daisy-chained drivers, the total maximum operating current limits at approximately **1.28 A**.
- **Microcontroller Load:** The ESP-32 consumes approximately **100 mA** during active Wi-Fi operation.

Based on these calculations, the discrete 5V buck converter is strictly required to handle the **>1 A** dynamic loads generated by the display matrix safely.

RESULTS AND DISCUSSION:

Hardware Implementation:

The primary outcome of this project is a fully functional 8x8 interactive 7-segment display matrix. The physical build relies on an ESP32 microcontroller managing an array of MAX7219 display drivers.

Code implementation:

MULTI MODULE INTEGRATION AND INTEFACE CONTROL

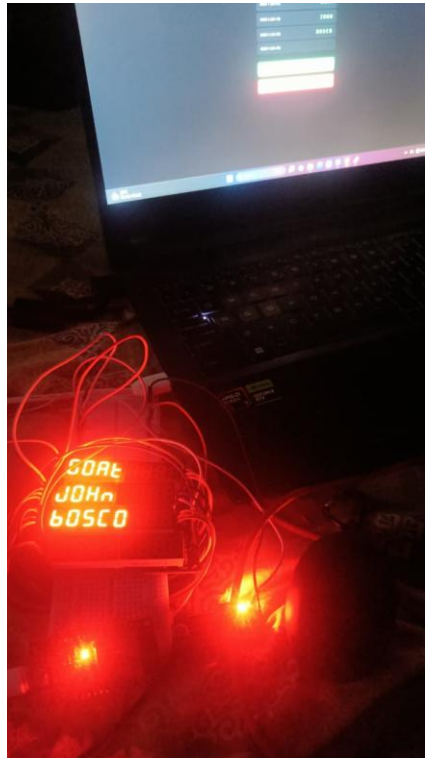


Fig.4. 4x8 integration

CODE:

```
#include <WiFi.h>
#include <WebServer.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES    4
#define CLK_PIN        18
#define DATA_PIN       23
#define CS_PIN          5

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN,
MAX_DEVICES);
WebServer server(80);

byte displayState[MAX_DEVICES][8];

const char* html = R"rawliteral(
<!DOCTYPE html><html><head><meta name='viewport' content='width=device-
width, initial-scale=1'>
```

```

<style>
body{background:#000;color:#0f0;text-align:center;font-family:sans-
serif;margin:0;padding:10px;}

h3{color:#fff;text-shadow:0 0 10px #0f0;}
.matrix{display:flex;flex-direction:column;gap:15px;align-items:center;}
.module-row{display:flex;flex-direction:row-reverse;gap:8px;border:1px
solid #333;padding:10px;background:#050505;border-radius:5px;}
.digit{position:relative;width:35px;height:55px;background:#111;border:1px
solid #222;}
.seg{position:absolute;background:#222;border-radius:2px;transition:0.2s;}
.seg.active{background:#f00;box-shadow:0 0 12px #f00;}
.s-6{top:4px;left:6px;width:22px;height:4px;}
.s-5{top:6px;right:3px;width:4px;height:18px;}
.s-4{bottom:6px;right:3px;width:4px;height:18px;}
.s-3{bottom:4px;left:6px;width:22px;height:4px;}
.s-2{bottom:6px;left:3px;width:4px;height:18px;}
.s-1{top:6px;left:3px;width:4px;height:18px;}
.s-0{top:25px;left:6px;width:22px;height:4px;}
.s-7{bottom:3px;right:-5px;width:6px;height:6px;border-radius:50%;}
button{margin:20px;padding:12px
25px;background:#b00;color:white;border:none;border-radius:5px;font-
weight:bold;cursor:pointer;}
</style></head>
<body>

```

```

<h3>4-Module Interactive Canvas</h3>
<div class="matrix" id="m"></div>
<button onclick="fetch('/clear');location.reload();">CLEAR ALL</button>

```

```

<script>
const m=document.getElementById('m');

for(let mod=0;mod<4;mod++){
  const row=document.createElement('div');
  row.className='module-row';

  for(let dig=0;dig<8;dig++){
    const dDiv=document.createElement('div');
    dDiv.className='digit';

    for(let s=0;s<8;s++){
      const sDiv=document.createElement('div');
      sDiv.className=`seg s-${s}`;

      sDiv.onpointerdown=e=>{
        e.preventDefault();
      }
    }
  }
}

```

```

        sDiv.classList.toggle('active');
        fetch(`/draw?m=${mod}&d=${dig}&s=${s}`);
    };

    dDiv.appendChild(sDiv);

}

row.appendChild(dDiv);
}

m.appendChild(row);
}
</script>
</body></html>
)rawliteral";

void updateHardware()
{
    for(int m=0;m<MAX_DEVICES;m++)
    {
        for(int i=0;i<8;i++)
        {
            mx.setColumn(m, i, displayState[m][i]);
        }
    }
}

void handleDraw()
{
    int m = server.arg("m").toInt();
    int d = server.arg("d").toInt();
    int s = server.arg("s").toInt();

    if(m < 0 || m >= MAX_DEVICES || d < 0 || d >= 8 || s < 0 || s > 7)
        return;

    displayState[m][s] ^= (1 << d);

    updateHardware();

    server.send(200, "text/plain", "OK");
}

void setup()
{
    Serial.begin(115200);

```

```

mx.begin();
mx.control(MD_MAX72XX::SHUTDOWN, MD_MAX72XX::OFF);
mx.control(MD_MAX72XX::INTENSITY, 5);
mx.clear();

WiFi.softAP("Smart-Display-Canvas", "12345678");

Serial.println("Access Point Started!");
Serial.print("IP Address: ");
Serial.println(WiFi.softAPIP());

server.on("/", []()
{
    server.send(200, "text/html", html);
});

server.on("/draw", handleDraw);

server.on("/clear", []()
{
    mx.clear();
    memset(displayState, 0, sizeof(displayState));
    server.send(200, "text/plain", "Cleared");
});

server.begin();
}

void loop()
{
    server.handleClient();
}

```

FINAL PHASE:

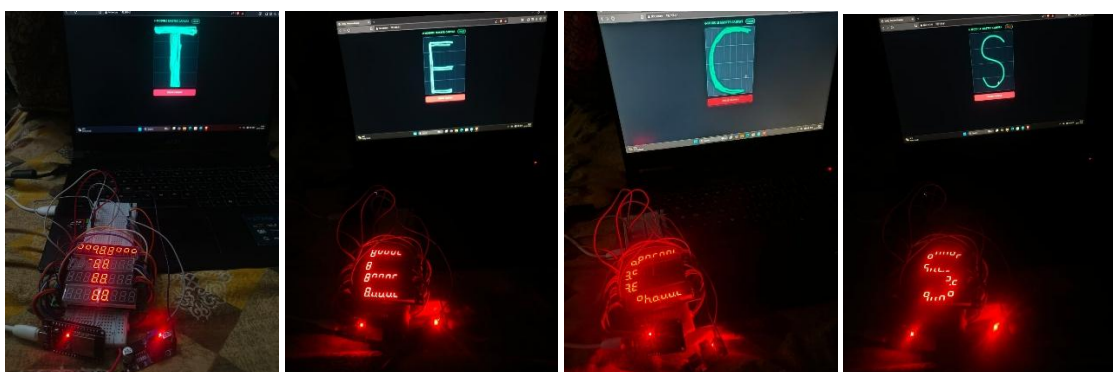


Fig.5. Final Output

CODE:

```
#include <WiFi.h>
#include <WebServer.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4
#define CLK_PIN 18
#define DATA_PIN 23
#define CS_PIN 5

MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN,
MAX_DEVICES);
WebServer server(80);

byte displayState[MAX_DEVICES][8];

const char* html = R"rawliteral(
<!DOCTYPE html><html><head><meta name='viewport' content='width=device-
width, initial-scale=1, maximum-scale=1, user-scalable=no'>
<style>
body{background:#000;color:#fff;font-family:sans-
serif;margin:0;display:flex;flex-direction:column;align-
items:center;touch-action:none;}
#whiteboard{width:280px;height:350px;background:#fff;border:5px solid
#333;margin-top:30px;position:relative;touch-action:none;border-
radius:10px;}
.btn{width:280px;padding:15px;background:#f00;color:#fff;border:none;borde
r-radius:5px;font-weight:bold;margin-top:20px;}
h3{color:#0f0;margin-top:20px;}
</style></head><body>
<h3>Vertical 4-Module Sync</h3>
<div id="whiteboard"></div>
<button class="btn" onclick="fetch('/clear'); location.reload();">ERASE
ALL</button>

<script>
const board=document.getElementById('whiteboard');
let lastTime=0;

board.onpointermove=(e)=>{
  if(e.buttons!==1)return;

  const now=Date.now();
  if(now-lastTime<35)return;
```

```

lastTime=now;

const rect=board.getBoundingClientRect();
const x=(e.clientX-rect.left)/rect.width;
const y=(e.clientY-rect.top)/rect.height;

if(x>=0&&x<=1&&y>=0&&y<=1){

    const m=Math.floor(y*4);
    const d=7-Math.floor(x*8);
    const ly=(y*4)%1;

    let sBit=-1;

    if(ly<0.45){
        const dy=ly*2.2;
        if(dy<x&&dy<(1-x))sBit=6;
        else if(x<dy&&x<(1-dy))sBit=1;
        else if(x>(1-dy))sBit=5;
        else sBit=0;
    }
    else if(ly>0.55){
        const dy=(ly-0.55)*2.2;
        if(dy<x&&dy<(1-x))sBit=0;
        else if(x<dy&&x<(1-dy))sBit=2;
        else if(x>(1-dy))sBit=4;
        else sBit=3;
    }
    else{
        sBit=0;
    }

    if(sBit!=-1){
        fetch(`/draw?m=${m}&d=${d}&s=${sBit}`);

        const ink=document.createElement('div');

        ink.style.cssText=`position:absolute;width:5px;height:5px;background:#000;
        left:${x*100}%;top:${y*100}%;border-radius:50%;pointer-events:none;`;
        board.appendChild(ink);
    }
}
};
</script>
</body></html>
)rawliteral";

```

```

void updateHardware()
{
  for(int m=0;m<MAX_DEVICES;m++)
  {
    for(int i=0;i<8;i++)
    {
      mx.setColumn(m,i,displayState[m][i]);
    }
  }
}

void handleDraw()
{
  int m=server.arg("m").toInt();
  int d=server.arg("d").toInt();
  int s=server.arg("s").toInt();

  if(m<0||m>=MAX_DEVICES||d<0||d>=8||s<0||s>7)return;

  displayState[m][s]|=(1<<d);

  updateHardware();

  server.send(200,"text/plain","OK");
}

void setup()
{
  Serial.begin(115200);

  mx.begin();
  mx.control(MD_MAX72XX::SHUTDOWN,MD_MAX72XX::OFF);
  mx.control(MD_MAX72XX::INTENSITY,2);
  mx.clear();

  WiFi.softAP("Vertical-Canvas","12345678");

  server.on("/",[]()
  {
    server.send(200,"text/html",html);
  });

  server.on("/draw",handleDraw);

  server.on("/clear",[]()
  {
    mx.clear();
  });
}

```

```

        memset(displayState,0,sizeof(displayState));
        server.send(200);
    });

    server.begin();
}

void loop()
{
    server.handleClient();
}

```

Interface Operation:

The system is programmed so that every individual LED segment can be controlled independently. Users interact with the display through a custom web page featuring a drawing canvas. To do this, a phone or laptop connects directly to the ESP32's local Wi-Fi network (running in Access Point mode). Any sketch drawn on the remote canvas is immediately mirrored onto the physical LED matrix.

System Connectivity

This outcome validates the project's data pipeline. It successfully demonstrates how input data is captured on a remote device, transmitted wirelessly to the ESP32, and then routed serially to the display modules for physical output.

CONCLUSION

The development and implementation of the Interface Matrix System were successfully completed using an ESP32 microcontroller and multiple MAX7219 LED matrix modules. The system enables real-time wireless interaction through a web-based interface, allowing users to draw and control the display using any Wi-Fi-enabled device.

REFERENCE

1. ESP32 datasheet link
https://drive.google.com/file/d/1YYEfnvFbxy2HQLfT4294Hevo23jeGdGm/view?usp=drive_link
2. MAX7219 datasheet link
https://drive.google.com/file/d/19JXIDpoE1UKZpfFHsclFvC_H47Maiv8i/view?usp=drive_link